

CSCE 636 DEEP LEARNING · MODULE 05 · LECTURE NOTES

Convolutional Networks

The architecture that made the case.

MODULE OUTCOMES

- Explain convolution as weight sharing plus locality.
- Compute receptive fields and reason about depth.
- Explain the standard architectural moves and why each was introduced.
- Explain what the learned filters represent.
- Explain where CNNs have been displaced and where they persist.

1 The operation

CONVOLUTION IS WEIGHT SHARING WITH A LOCALITY CONSTRAINT

Locality: each output value depends only on a small spatial neighbourhood of the input, not on the whole image. This encodes the assumption that what matters about a pixel is determined nearby.

Weight sharing: the same small filter is applied at every spatial position, so a feature learned at one location is detectable at all of them — and the parameters do not grow with image size.

Together these reduce the parameter count by orders of magnitude relative to a fully connected layer and build in *translation equivariance*: translate the input and the feature map translates identically.

Note equivariance, not invariance. The feature map *moves* with the input; it does not stay the same. Pooling, striding, and eventually global pooling introduce approximate invariance, and convolution alone does not. **Conflating the two is a common and consequential error** — it leads people to expect a CNN to be robust to translation when the architecture only guarantees that the representation shifts predictably, and the robustness actually comes from pooling and augmentation.

$$r_l = r_{l-1} + (k_l - 1) \cdot \prod_{j < l} s_j$$

The receptive field of a unit — the region of the input it can possibly depend on — **grows linearly with depth and multiplicatively with stride**. Ten 3×3 layers at stride 1 give a 21×21 receptive field, which is small for a 224×224 image; **adding a stride-2 layer every other block makes it grow far faster, which is why downsampling appears in every architecture**. A unit cannot use information outside its **receptive field** — so if the object to be recognised is larger than the receptive field at the layer where the decision is made, **the network literally cannot see the object**, and no amount of training fixes it. Receptive field arithmetic is therefore a design constraint rather than an analysis, and it explains several architecture choices directly.

2 The architectural moves

Idea	What it fixed	Note
Stacks of 3×3 filters	Large filters are expensive — a 7×7 filter has 49 parameters per channel pair.	Two stacked 3×3 layers have the same 5×5 receptive field with fewer parameters and an extra nonlinearity. VGG's contribution, and it has stuck.
1×1 convolution	Channel counts exploding as depth increases.	Mixes channels without touching space — it is a per-pixel linear layer, and it makes bottleneck blocks possible.
Residual blocks	Vanishing gradients at depth.	Enabled networks beyond a hundred layers (Module 02 §3). The most important single entry in this table.
Batch normalisation	Training instability and slow convergence.	Permitted much larger learning rates (Module 04 §3).
Depthwise separable convolution	Parameter and compute cost on mobile hardware.	Factors a convolution into a per-channel spatial filter and a 1×1 mixing step — roughly 8× cheaper for comparable accuracy. MobileNet, and now widespread.
Global average pooling	An enormous fully connected head — AlexNet's dense layers held most of its parameters.	Averages each channel to a single number, replacing millions of parameters with none and improving generalisation.
Dilated (atrous) convolution	Growing the receptive field without losing resolution.	Segmentation and dense prediction, where downsampling destroys exactly what the output needs.

WHAT THE FILTERS LEARN, AND WHY IT MATTERS

Early layers learn oriented edge and colour-opponent detectors that look strikingly like Gabor filters — and strikingly like the receptive fields measured in mammalian primary visual cortex decades earlier. **Nobody put them there.**

Middle layers respond to textures and object parts — fur, mesh patterns, wheels, eyes — and late layers to whole objects and configurations.

This hierarchy was not designed; it emerged from the objective, and it appears consistently across architectures, datasets, and training runs. It is one of the more genuinely surprising empirical results in the field.

And it is the basis of transfer learning (Module 09): **early features are general** — edges are edges regardless of the task — **so they are worth reusing**, while late features are task-specific and are what fine-tuning replaces. The practical importance of this finding is hard to overstate; it is why a model trained on ImageNet is a useful starting point for medical imaging.

3 Where CNNs stand now

TRANSFORMERS DISPLACED CNNs AT SCALE, NOT EVERYWHERE

Vision transformers outperform CNNs given enough data — on the order of hundreds of millions of images — because attention's weaker inductive bias becomes an *advantage* once the data can supply what the bias would otherwise have assumed, and can discover relationships the bias would have excluded.

Below that scale, CNNs remain competitive or better. This is exactly Module 01 §2's argument: **a correct prior substitutes for data, and the substitution stops mattering when data is abundant** — so the crossover point is a data-quantity question, not an architectural one.

And modern CNNs substantially narrowed the gap. ConvNeXt took a ResNet and applied the training recipe developed for transformers — the augmentation, the optimiser, the schedule, the normalisation choices — and matched transformer performance without changing the fundamental operation.

So much of the reported architectural advantage was training recipe rather than architecture. This is a recurring finding in this field, and it is a strong reason to be sceptical of architecture comparisons where the training procedures differ — which they almost always do, because each paper tunes its own method and inherits the baseline's hyperparameters from an older paper.

- **Limited data** — the inductive bias is worth the constraint, which covers most applied problems outside the largest labs.
- **Limited compute or tight latency** — convolution is cheaper per unit of accuracy, and a decade of hardware and compiler work has been aimed at it specifically (CSCE 735 Module 09).
- **High resolution** — **attention is quadratic in token count while convolution is linear in pixels**, so a 4K image is a very different proposition for the two.
- **Dense prediction** — segmentation, depth estimation, optical flow, denoising — where the output has spatial structure and resolution must be preserved.
- **And on-device inference**, where depthwise separable convolutions, integer quantisation (Module 13), and dedicated accelerators are all mature for convolution and less so for attention. **Most deployed vision models are still convolutional**, which the research literature does not reflect and which is worth knowing when reading it.

4 The graphics connection

CONVOLUTION IN THE RENDERING PIPELINE

Learned denoising (CSCE 647 Module 12) is a convolutional network operating on a sparsely-sampled noisy render together with auxiliary geometry buffers — surface normals, albedo, depth, motion vectors. **It made path tracing interactive**, which a decade of sampling improvements had not.

Super-resolution and frame generation — DLSS and its competitors — are likewise convolutional, and are now a standard part of the real-time pipeline rather than an experiment.

And both exploit precisely the locality assumption: a pixel's correct value depends on its spatial neighbourhood and on the geometric buffers at that neighbourhood — **which is genuinely true of rendering in a way it is not true of arbitrary data**, because the rendering equation itself is local in screen space given the geometry.

So the architecture matches the domain, which is Module 01 §2 again, **and it is why these methods worked almost immediately once anyone tried them** — the prior was correct, the auxiliary buffers were already being computed, and the training data could be generated in unlimited quantity by rendering the same scene at high sample counts.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Stanford CS231n — convolutional networks notes (free)	The reference for §1 and §2 , including the receptive field arithmetic and the architecture history.
He et al. — Deep Residual Learning; Howard et al. — MobileNets (free)	Two of §2's rows, from their papers. The depthwise separable factorisation is worth deriving once.
Liu et al. — A ConvNet for the 2020s (ConvNeXt) (free)	The §3 result . A step-by-step modernisation of a ResNet, with each change measured — which is also a model of how to ablate.
Chaitanya et al. — Interactive Reconstruction of Monte Carlo Image Sequences (free)	The §4 denoiser , and the connection back to CSCE 647.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement 2D convolution from scratch and verify against the framework.

- 2 **Count the parameters** of a convolutional and a fully connected layer with identical input and output shapes.
- 3 **Compute the receptive field** of each layer in a network you use, and compare against the object sizes in your data.
- 4 **Demonstrate equivariance and the lack of invariance:** shift an input and compare the feature maps, then compare the final predictions.
- 5 Replace one 5×5 layer with two 3×3 and compare parameters, compute, and accuracy.
- 6 **Implement a depthwise separable convolution** and measure the saving.
- 7 Replace a fully connected head with global average pooling and report the parameter reduction.
- 8 **Visualise the first-layer filters** of a trained network.
- 9 Visualise what maximally activates units at three different depths.
- 10 **Train a CNN and a vision transformer** on the same small dataset and report which wins and why.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What two ideas is convolution, and what does each buy?
- 2 Distinguish equivariance from invariance and say where each comes from.
- 3 Compute a receptive field and say why it constrains architecture.
- 4 Give seven architectural moves and the problem each fixed.
- 5 What do early, middle, and late filters learn, and why does that matter?
- 6 When do transformers beat CNNs, and what explains the crossover?
- 7 What did ConvNeXt show?
- 8 Give five places convolution still wins.
- 9 How is convolution used in the rendering pipeline?